Teoría de Algoritmos

(TB024) Curso Echevarría

# Trabajo Práctico 1

20 de abril de 2026


| Integrantes | Padrón |
|---|---|
| Joaquín Hernández | 110470 |
| Juan Francisco Cuevas | 107963 |
| Alexander Terrussi | 00 |
| Dante Finci | 00 |
| Víctor Zacarías | 107080 |

# Índice

**1. Comparación de Monedas** **3**
1.1. Introducción y Problema . . . . . 3
1.2. Algoritmo de División y Conquista . . . . . 3
1.3. Complejidad temporal y cantidad de pesadas . . . . . 3
1.4. Metodología experimental . . . . . 4
1.5. Resultados y gráficos . . . . . 4
1.6. Relación entre tiempo de ejecución y pesadas . . . . . 4

**2. Algoritmo Paolo** **5**
2.1. Introducción algoritmo Paolo . . . . . 5

**3. Aspiradora con Backtracking** **8**
3.1. Introducción y Problema . . . . . 8

**4. Descomposición de Cadena con Programación Dinámica** **9**
4.1. Introducción y Problema . . . . . 9

# 1. Comparación de Monedas

## 1.1. Introducción y Problema

Se tiene una bolsa con $n$ monedas de igual denominación, de las cuales exactamente una es falsa y es más liviana que el resto. El objetivo es identificar la moneda falsa sin ordenar por peso y usando División y Conquista.

La entrada se representa como una lista de pesos en un archivo de texto con valores separados por comas. Se asume que todos los pesos verdaderos son iguales (por ejemplo 10) y la falsa es más liviana (por ejemplo 4).

## 1.2. Algoritmo de División y Conquista

El algoritmo divide el arreglo en mitades. En cada paso compara las monedas de las posiciones centrales:

1. Si las dos monedas centrales son distintas, la más liviana es la falsa.
2. Si son iguales, se recurre sobre ambas mitades y luego se elige el mínimo entre los resultados.

Cada comparación de dos monedas se contabiliza como una pesada. La función principal parsea el archivo, llama al algoritmo y reporta tiempo y cantidad de pesadas.

El código del algoritmo de División y Conquista es:

```python
def encontrar_moneda_falsa_dyc(monedas, inicio, fin, contador):
  if inicio >= fin:
    return monedas[inicio]

  if fin - inicio == 1:
    contador[0] += 1
    return monedas[inicio] if monedas[inicio] <= monedas[fin] else monedas[fin]

  medio = (inicio + fin) // 2

  contador[0] += 1
  if monedas[medio] != monedas[medio + 1]:
    return monedas[medio] if monedas[medio] < monedas[medio + 1] else monedas[medio
     + 1]

  izquierdo = encontrar_moneda_falsa_dyc(monedas, inicio, medio, contador)
  derecho = encontrar_moneda_falsa_dyc(monedas, medio + 1, fin, contador)

  contador[0] += 1
  return izquierdo if izquierdo <= derecho else derecho
```

## 1.3. Complejidad temporal y cantidad de pesadas

Sea $T(n)$ el tiempo del algoritmo. La recurrencia es:

$$T(n) = 2T(n/2) + O(1)$$

La ecuación se justifica porque en cada llamada se divide el problema en dos subproblemas de tamaño $n/2$ (dos llamadas recursivas) y el costo adicional de comparar y seleccionar resultados es constante, es decir, $O(1)$.

Por el Teorema Maestro, $Complejidad = O(n)$.

## 1.4. Metodología experimental

Se evaluó el algoritmo con tamaños $n = \{100, 500, 1000, 5000, 10000, 50000\}$. Para cada tamaño se generó un archivo con una única moneda falsa y se ejecutó:

```
python3 ejercicios/ej-dyc/monedas.py 100
```

El tiempo se midió con la función time del sistema y se registró la cantidad de pesadas. Los resultados se guardaron en una tabla y se graficaron junto con una curva teórica lineal $O(n)$.

## 1.5. Resultados y gráficos

| $n$ | Tiempo (s) | Pesadas |
|---|---|---|
| 100 | 0.000021 | 160 |
| 500 | 0.000080 | 746 |
| 1000 | 0.000162 | 1507 |
| 5000 | 0.000904 | 7945 |
| 10000 | 0.001782 | 15900 |
| 50000 | 0.010770 | 82764 |

Cuadro 1: Resultados experimentales del ejercicio 1.

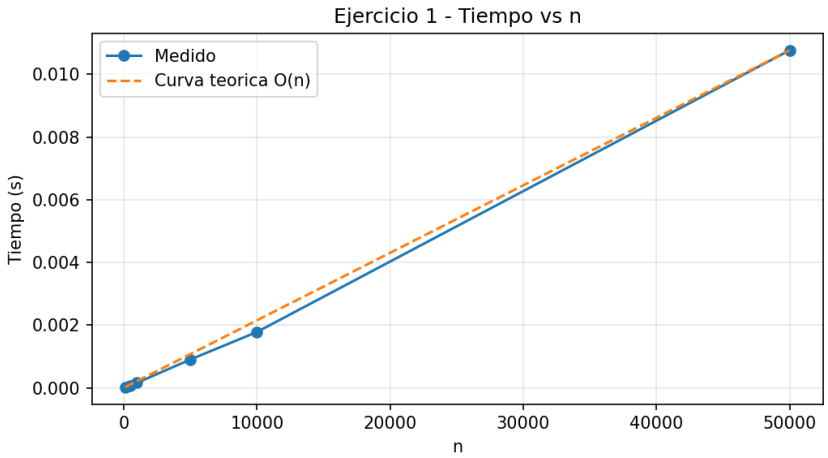


Figura 1: Tiempo de ejecución medido vs curva teórica $O(n)$.

Los gráficos muestran un crecimiento cercano al lineal, consistente con la complejidad analizada. La curva teórica se escaló para superponerla con la serie medida.

## 1.6. Relación entre tiempo de ejecución y pesadas

Es razonable vincular el tiempo con la cantidad de pesadas porque cada pesada implica una comparación de pesos, y el algoritmo realiza un número lineal de comparaciones. Sin embargo, el tiempo también incluye costos de lectura de datos y llamadas recursivas, por lo que la relación no es perfectamente proporcional. Aun así, ambos indicadores reflejan el mismo orden de crecimiento $O(n)$.

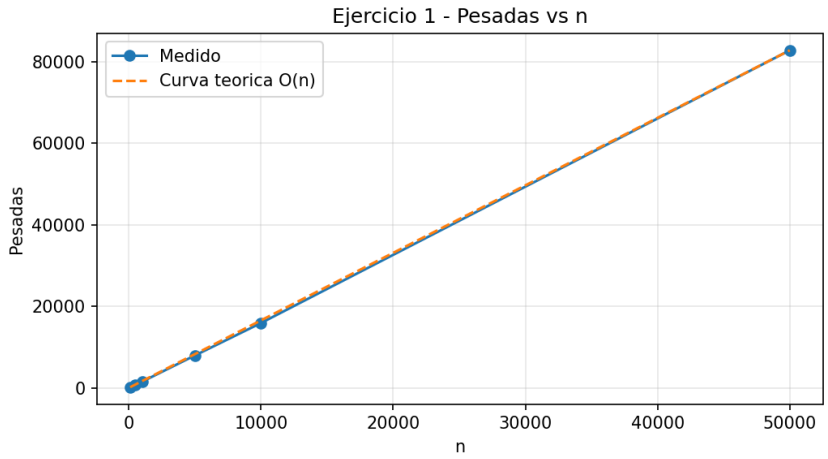


Figura 2: Cantidad de pesadas medida vs curva teórica $O(n)$.

# 2. Algoritmo Paolo

## 2.1. Introducción algoritmo Paolo

El algoritmo "paolo", encontrado por el equipo de antropologia escrito en un extraño lenguaje es el siguiente:

```
10 CLS
20 INPUT "INGRESE EL NUMERO DE VERTICES "; N
30 PRINT
40 DIM COST (N,N)
50 DIM DIST (N)
60 DIM SOL (N)
70 A=1
80 PRINT "INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)"
90 PRINT
100 INPUT "EL EJE "; A, B
110 IF A=0 THEN 140
120 INPUT "COSTO DEL EJE "; COST(A,B)
130 GOTO 100
140 FOR I=1 TO N
150 FOR J=1 TO N
160 IF COST (I,J)=0 THEN COST (I,J)=15000
170 NEXT J
180 NEXT I
190 FOR I=1 TO N
200 FOR J=1 TO I
210 COST (I,J)=COST (J,I)
220 NEXT J
230 NEXT I
240 INPUT "INGRESE EL VERTICE DE SALIDA "; V
250 FOR I=1 TO N
260 DIST(I)=COST(V,I)
270 SOL(I)=0
280 NEXT I
290 GOSUB 1000
300 PRINT "SALIDA","LLEGADA","DISTANCIA"
310 FOR I=1 TO N
320 IF DIST(I)<15000 THEN PRINT V, I, DIST(I)
330 NEXT I
340 INPUT "OTRA VEZ? (SI/NO) "; RES$
350 IF RES$="NO" THEN END
360 FOR I=1 TO N
370 SOL(I)=0: DIST(I)=0
380 NEXT I
```

```
390 GOTO 240
1000 SOL(V)=1
1010 DIST(V)=0
1020 FOR I=1 TO N-1
1030 U=15000
1040 FOR J=1 TO N
1050 IF DIST(J)<=U AND SOL(J)=0 THEN U=J
1060 NEXT J
1070 SOL(U)=1
1080 FOR J=1 TO N
1090 IF DIST(J)>= (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
1100 NEXT J
1110 NEXT I
1120 RETURN
```

Listing 1: Algoritmo Paolo original

Su traduccion mas literal a un lenguaje moderno seria así:

```

def paolo():
    n = int(input("INGRESE EL NUMERO DE VERTICES "))
    cost = [[0 for _ in range(n)] for _ in range(n)]
    dist = [0 for _ in range(n)]
    sol = [0 for _ in range(n)]
    a = 1
    print("INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)")
    print()
    while True:
        a, b = map(int, input("EL EJE ").split())
        if a == 0 and b == 0:
            break
        cost[a][b] = int(input("COSTO DEL EJE "))
    for i in range(n):
        for j in range(n):
            if cost[i][j] == 0:
                cost[i][j] = 15000
    for i in range(n):
        for j in range(i):
            cost[i][j] = cost[j][i]
    v = int(input("INGRESE EL VERTICE DE SALIDA "))
    for i in range(n):
        dist[i] = cost[v][i]
        sol[i] = 0
    paolo_sub(n, cost, dist, sol, v)
    print("SALIDA", "LLEGADA", "DISTANCIA")
    for i in range(n):
        if dist[i] < 15000:
            print(v, i, dist[i])
    res = input("OTRA VEZ? (SI/NO) ")
    if res == "NO":
        return
    for i in range(n):
        sol[i] = 0
        dist[i] = 0
    paolo()

def paolo_sub(n, cost, dist, sol, v):
    sol[v] = 1
    dist[v] = 0
    for i in range(n - 1):
        u = 15000
        for j in range(n):
            if dist[j] <= u and sol[j] == 0:
                u = j
        sol[u] = 1
        for j in range(n):
            if dist[j] >= (dist[u] + cost[u][j]):
                dist[j] = dist[u] + cost[u][j]

if __name__ == "__main__":
```

```
paolo()
```

Listing 2: Algoritmo Paolo traducido

Nuestro objetivo en esta seccion es analizar el algoritmo y responder las siguientes consignas:

1. ¿Qué problema resuelve el código encontrado?
2. Algoritmo. ¿Qué algoritmo conocido replica dicho código?
3. Estructuras de datos. ¿Cuál es el uso de las principales estructuras de datos del programa?
4. ¿Por qué se puede considerar que es un algoritmo Geedy? ¿Cuál es la regla que se aplica?
5. Complejidad temporal y espacial. ¿Cuál es el orden de complejidad temporal y espacial del programa? NOTA: no se requiere graficar, sólo una interpretación teórica.

# 3. Aspiradora con Backtracking

## 3.1. Introducción y Problema

Describir el problema de la aspiradora y el enfoque de Backtracking.

# 4. Descomposición de Cadena con Programación Dinámica

## 4.1. Introducción y Problema

Describir el problema de descomposición de cadenas y el enfoque de Programación Dinámica.